Analytics & Big Data

Session 7: Machine learning II

Prof. Dr. Gerit Wagner

(2026-04-20)

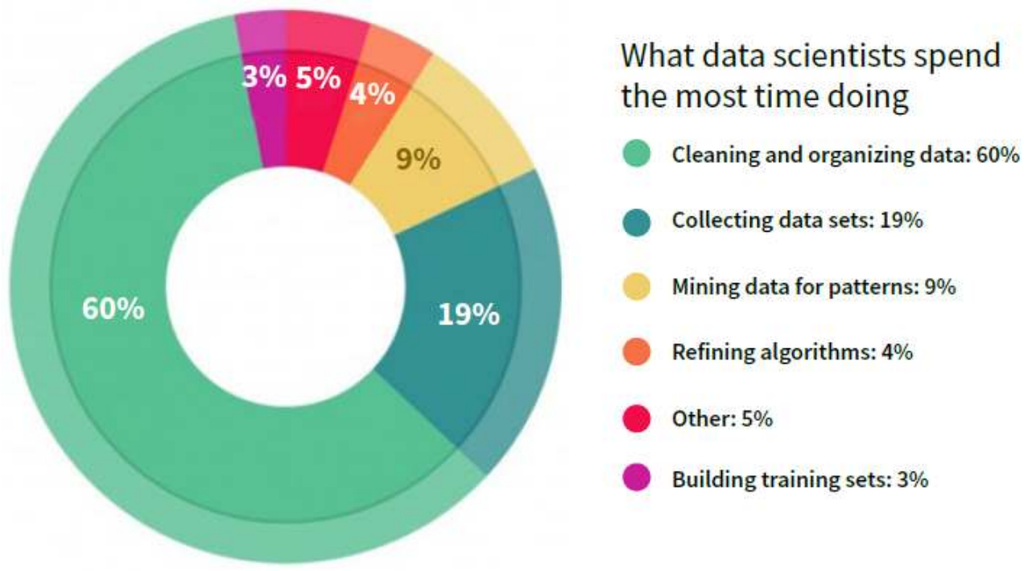
Learning objectives

- A
- B
- C

TODO

Data Preparation and Enrichment

> The data collection and preparation phase is the most labor-intensive one, consuming on average between 60-80% of a data scientist's time. It's critical therefore to select a tool that can automate or at least speed the workflows associated with data preparation.



2016 Dataiku, Inc.

Data Cleaning

1. Proof of correctness of the data

- examine for irregular outliers (e.g. Age=236)
- examine for typographical errors (e.g. Frankfrut)
- examine for different writing styles (e.g. behavior/behaviour)
-

2. Handling missing values

eliminate
- all objects having missing values (rowwise)
- variables having missing values first, then remaining objects

replace
- with the true values
- with mean values
- imputing by estimated values (e.g. using k-Nearest Neighbors)

Missing Values Strategies

Initial Situation

Variable 1	Variable 2	Variable 3	Variable 4	Variable 5
8.95	0.99	NA	370.53	-34.97
NA	NA	168.02	617.46	-99.49
18.38	0.48	58.06	1177.66	-44.21
16.10	NA	35.39	NA	77.35
23.14	NA	230.44	408.97	-71.79
24.77	0.92	206.83	NA	46.13
20.72	NA	228.74	659.37	-67.77
15.52	0.04	22.46	NA	36.63

Smart Strategy

Step 1:

Variable 1	Variable 2	Variable 3	Variable 4	Variable 5
8.95	0.99	NA	370.53	-34.97
NA	NA	168.02	617.46	-99.49
18.38	0.48	58.06	1177.66	-44.21
16.10	NA	35.39	NA	77.35
23.14	NA	230.44	408.97	-71.79
24.77	0.92	206.83	NA	46.13
20.72	NA	228.74	659.37	-67.77
15.52	0.04	22.46	NA	36.63

Step 2:

Variable 1	Variable 3	Variable 5
~~8.95~~	~~NA~~	~~-34.97~~
~~NA~~	~~168.02~~	~~-99.49~~
18.38	58.06	-44.21
16.10	35.39	77.35
23.14	230.44	-71.79
24.77	206.83	46.13
20.72	228.74	-67.77
15.52	22.46	36.63

Stupid Strategy

Variable 1	Variable 2	Variable 3	Variable 4	Variable 5
~~8.95~~	~~0.99~~	~~NA~~	~~370.53~~	~~-34.97~~
~~NA~~	~~NA~~	~~168.02~~	~~617.46~~	~~-99.49~~
18.38	0.48	58.06	1177.66	-44.21
~~16.10~~	~~NA~~	~~35.39~~	~~NA~~	~~77.35~~
~~23.14~~	~~NA~~	~~230.44~~	~~408.97~~	~~-71.79~~
~~24.77~~	~~0.92~~	~~206.83~~	~~NA~~	~~46.13~~
~~20.72~~	~~NA~~	~~228.74~~	~~659.37~~	~~-67.77~~
~~15.52~~	~~0.04~~	~~22.46~~	~~NA~~	~~36.63~~

Sampling

A population can be defined as including all people or items with the characteristic one wishes to understand.

Sampling is about to find a representative subset of that population.

Data represents the traces of the real-world processes, and exactly which traces we gather are decided by our sampling method.

There are two sources of randomness and uncertainty:

- the randomness and uncertainty underlying the process itself, and
- the uncertainty associated with the underlying sampling method.

Sampling in Times of Big Data

Question:

Is there any need for sampling in times of Big Data? Why not “N=ALL”?

Answer:

Data is not objective! Data does not speak for itself. Data is just a quantitative echo of the events of our society.

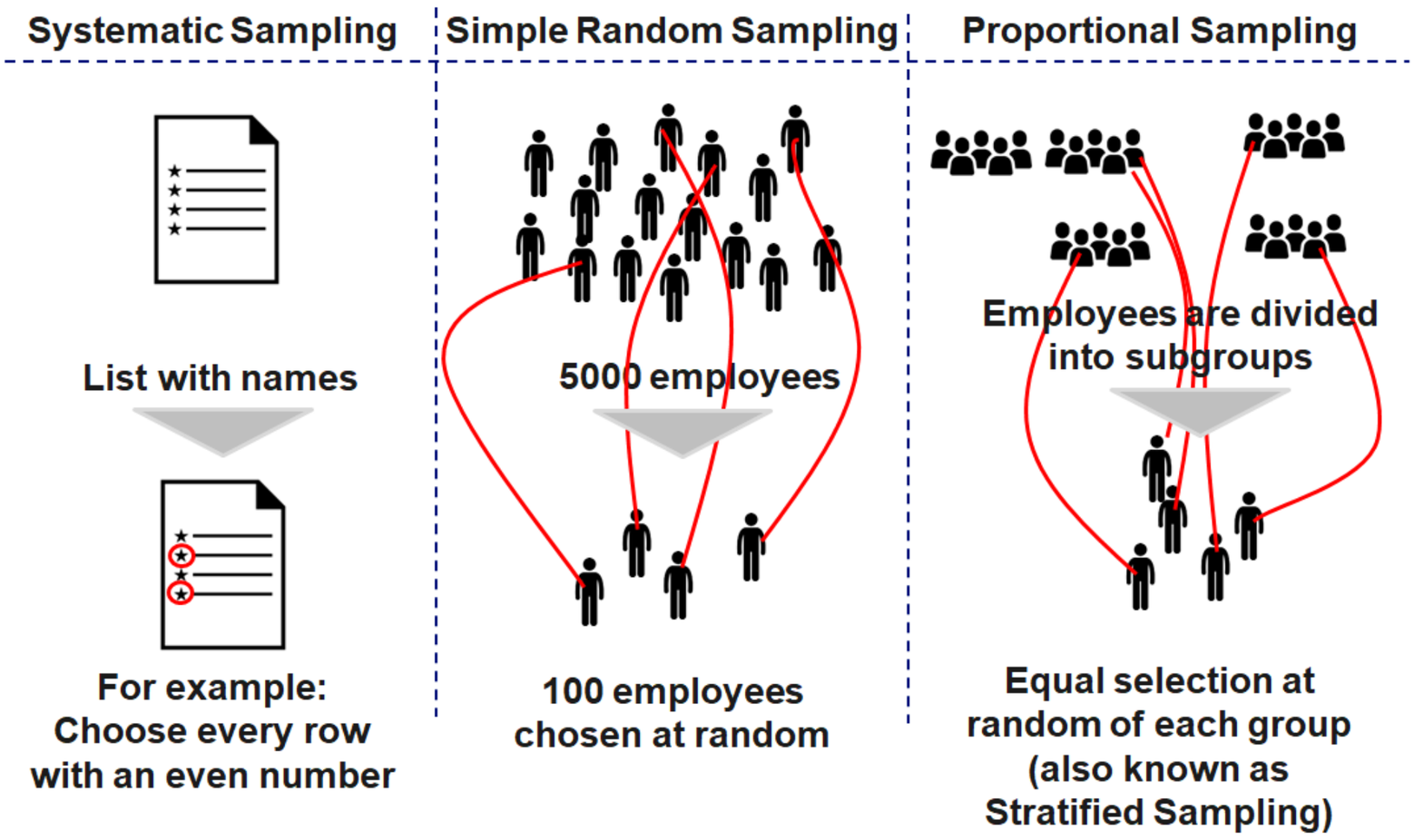
Examples:

- When analyzing the probability of customers terminating the relationship, a very small proportion of terminating customers (e.g. 0.2%) on the whole may result in a bias.
- When analyzing political attitudes via social media data, there might be a bias if people with specific attitudes are posting more frequently.

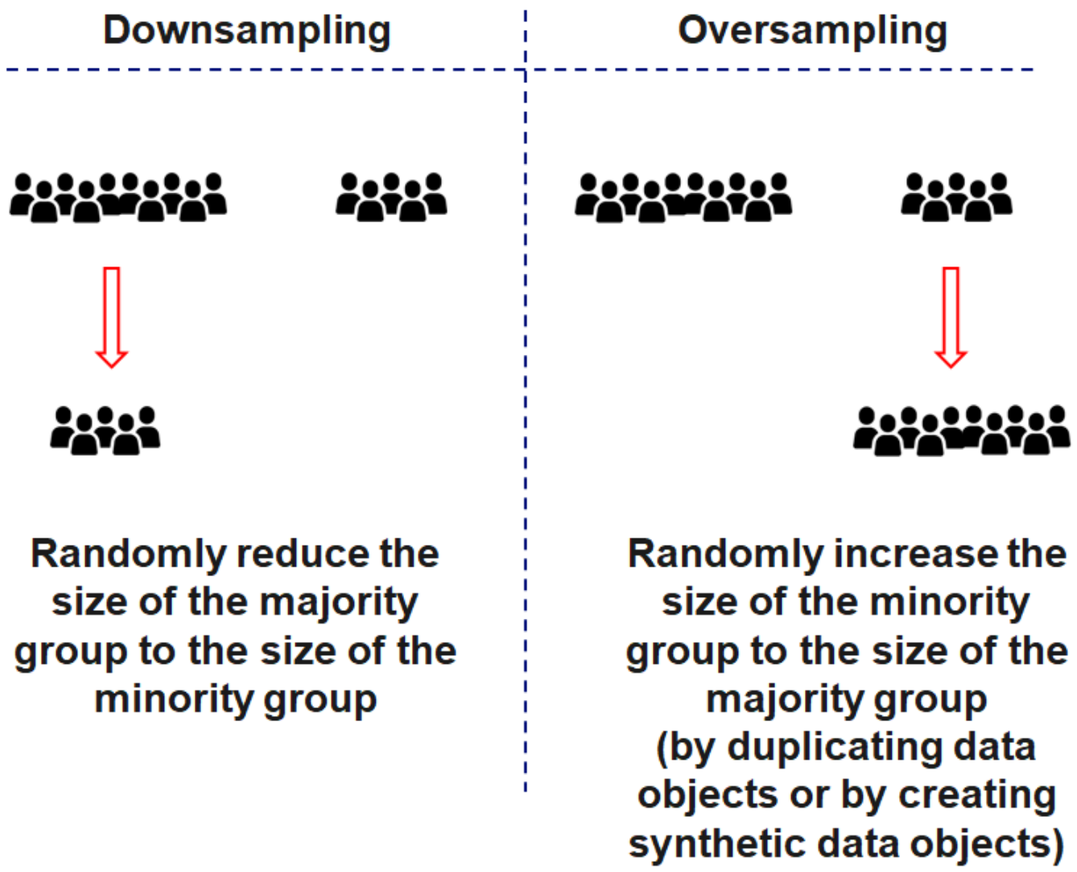
Reasons for Sampling

- The volume of data is too large to capture and process
- Design the analytics process using a subset of the data for performance reasons. Later use the complete data set.
- The data set doesn’t perfectly represent the target population.
- The data set is imbalanced.
- Use sampling to partition into training and test data.
- …

Popular Methods of Sampling (I)



Popular Methods of Sampling (II)



Systematic Sampling

Choose every row with an even number

day.mins	day.calls	day.charge	intl.mins	intl.calls	intl.charge	churn
157	79	26.69	7.1	6	1.92	No
184.5	97	31.37	8.7	4	2.35	No
258.6	84	43.96	11.2	5	3.02	No
129.1	137	21.95	12.7	6	3.43	Yes
187.7	127	31.91	9.1	5	2.46	No
128.8	96	21.9	11.2	2	3.02	No
156.6	88	26.62	12.3	5	3.32	No
120.7	70	20.52	13.1	6	3.54	No
332.9	67	56.59	5.4	9	1.46	Yes
196.4	139	33.39	13.8	4	3.73	No
190.7	114	32.42	8.1	3	2.19	No
189.7	66	32.25	10	5	2.7	No
224.4	90	38.15	13	2	3.51	No
155.1	117	26.37	10.6	4	2.86	No
62.4	89	10.61	5.7	6	1.54	Yes
84.8	95	14.42	14.2	6	3.83	No
226.1	105	38.44	10.3	5	2.78	No
212	121	36.04	12.6	10	3.4	No
249.6	118	42.43	11.8	3	3.19	Yes
176.8	94	30.06	8.3	4	2.24	No
140.4	94	23.87	11.1	9	3	No
126.3	102	21.47	9.4	2	2.54	No
173.1	85	29.43	14.6	15	3.94	Yes
124.8	82	21.22	10	4	2.7	No
85.8	77	14.59	9.2	4	2.48	No

Yes = 1

No = 11

Random Sampling

Creates totally random samples

Example: 40%

day.mins	day.calls	day.charge	intl.mins	intl.calls	intl.charge	churn
157	79	26.69	7.1	6	1.92	No
184.5	97	31.37	8.7	4	2.35	No
258.6	84	43.96	11.2	5	3.02	No
129.1	137	21.95	12.7	6	3.43	Yes
187.7	127	31.91	9.1	5	2.46	No
128.8	96	21.9	11.2	2	3.02	No
156.6	88	26.62	12.3	5	3.32	No
120.7	70	20.52	13.1	6	3.54	No
332.9	67	56.59	5.4	9	1.46	Yes
196.4	139	33.39	13.8	4	3.73	No
190.7	114	32.42	8.1	3	2.19	No
189.7	66	32.25	10	5	2.7	No
224.4	90	38.15	13	2	3.51	No
155.1	117	26.37	10.6	4	2.86	No
62.4	89	10.61	5.7	6	1.54	Yes
84.8	95	14.42	14.2	6	3.83	No
226.1	105	38.44	10.3	5	2.78	No
212	121	36.04	12.6	10	3.4	No
249.6	118	42.43	11.8	3	3.19	Yes
176.8	94	30.06	8.3	4	2.24	No
140.4	94	23.87	11.1	9	3	No
126.3	102	21.47	9.4	2	2.54	No
173.1	85	29.43	14.6	15	3.94	Yes
124.8	82	21.22	10	4	2.7	No
85.8	77	14.59	9.2	4	2.48	No

Yes = 1

No = 9

Proportional Sampling

Creates proportional samples

Example: 40%

day.mins	day.calls	day.charge	intl.mins	intl.calls	intl.charge	churn
157	79	26.69	7.1	6	1.92	No
184.5	97	31.37	8.7	4	2.35	No
258.6	84	43.96	11.2	5	3.02	No
129.1	137	21.95	12.7	6	3.43	Yes
187.7	127	31.91	9.1	5	2.46	No
128.8	96	21.9	11.2	2	3.02	No
156.6	88	26.62	12.3	5	3.32	No
120.7	70	20.52	13.1	6	3.54	No
332.9	67	56.59	5.4	9	1.46	Yes
196.4	139	33.39	13.8	4	3.73	No
190.7	114	32.42	8.1	3	2.19	No
189.7	66	32.25	10	5	2.7	No
224.4	90	38.15	13	2	3.51	No
155.1	117	26.37	10.6	4	2.86	No
62.4	89	10.61	5.7	6	1.54	Yes
84.8	95	14.42	14.2	6	3.83	No
226.1	105	38.44	10.3	5	2.78	No
212	121	36.04	12.6	10	3.4	No
249.6	118	42.43	11.8	3	3.19	Yes
176.8	94	30.06	8.3	4	2.24	No
140.4	94	23.87	11.1	9	3	No
126.3	102	21.47	9.4	2	2.54	No
173.1	85	29.43	14.6	15	3.94	Yes
124.8	82	21.22	10	4	2.7	No
85.8	77	14.59	9.2	4	2.48	No

Yes = 2

No = 8

Downsampling

Creates balanced samples

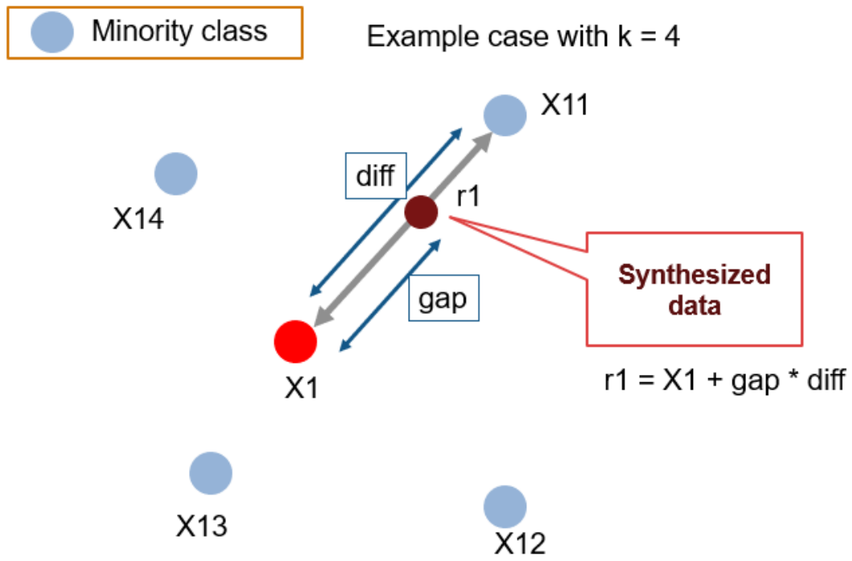
day.mins	day.calls	day.charge	intl.mins	intl.calls	intl.charge	churn
157	79	26.69	7.1	6	1.92	No
184.5	97	31.37	8.7	4	2.35	No
258.6	84	43.96	11.2	5	3.02	No
129.1	137	21.95	12.7	6	3.43	Yes
187.7	127	31.91	9.1	5	2.46	No
128.8	96	21.9	11.2	2	3.02	No
156.6	88	26.62	12.3	5	3.32	No
120.7	70	20.52	13.1	6	3.54	No
332.9	67	56.59	5.4	9	1.46	Yes
196.4	139	33.39	13.8	4	3.73	No
190.7	114	32.42	8.1	3	2.19	No
189.7	66	32.25	10	5	2.7	No
224.4	90	38.15	13	2	3.51	No
155.1	117	26.37	10.6	4	2.86	No
62.4	89	10.61	5.7	6	1.54	Yes
84.8	95	14.42	14.2	6	3.83	No
226.1	105	38.44	10.3	5	2.78	No
212	121	36.04	12.6	10	3.4	No
249.6	118	42.43	11.8	3	3.19	Yes
176.8	94	30.06	8.3	4	2.24	No
140.4	94	23.87	11.1	9	3	No
126.3	102	21.47	9.4	2	2.54	No
173.1	85	29.43	14.6	15	3.94	Yes
124.8	82	21.22	10	4	2.7	No
85.8	77	14.59	9.2	4	2.48	No

Yes = 5

No = 5

SMOTE

SMOTE (Synthetic Minority Oversampling Technique) is an oversampling technique where the synthetic samples are generated for the minority class. At first the total number of oversampling observations N is set up. Usually, it is selected such that the resulting class distribution is 1:1. Now, the iteration starts by first selecting a minority class instance at random. Next, the k nearest neighbors for that instance are obtained. For every neighbor calculate the difference as distance and multiply this difference by a random value between 0 and 1. Adding the result to the chosen instance creates a new synthetic instance. This is done until the number of needed instances is reached.



— Source: https://github.com/minoue-xx/Oversampling-Imbalanced-Data

Feature Engineering

Feature engineering is the process of using domain knowledge of the data to create features that make machine learning algorithms work. If feature engineering is done correctly, it increases the predictive power of machine learning algorithms by creating features from raw data that help facilitate the machine learning process.

A feature (variable, attribute) is depicted by a column in a dataset. Considering a generic two-dimensional dataset, each observation is depicted by a row and each feature by a column, which will have a specific value for an observation:

Columns (Features)

Rows (Observations)

	Feature 1	Feature 2	Feature 3
0	18.38	58.06	-44.21
1	16.10	35.39	77.35
2	23.14	230.44	-71.79
3	24.77	206.83	46.13

Features can be of two major types. **Raw features** are obtained directly from the dataset with no extra data manipulation or engineering. **Derived features** are usually obtained from feature engineering, where we extract features from existing data attributes. A simple example would be creating a new feature “Age” from an employee dataset containing “Birthdate”.

— Source: Sarkar, D.: Understanding Feature Engineering, towardsdatascience.com and Shekhar, A.: What Is Feature Engineering for Machine Learning?, medium.com.

Variants of Feature Engineering

1. Transformation
   - convert features (e.g., birth date → age)
   - build lag structures (e.g., time-lags)
   - normalization / standardization / scaling

2. Type Conversion

- if numerical type is needed, transform categorical into numerical data using dummy features
- if categorical type is needed or more informative, discretize numerical features (e.g., income → poor / rich classes)

3. Feature Combination

- create interaction features (e.g., school_score = num_schools × median_school with num_schools = number of schools within 5 miles of a property and median_school = median quality score of those schools)
- combine categories (e.g., when there are very few observations or too many dummy features)

4. Feature Composition
   - build ratios (e.g., returns from prices)
   - Principal Component Analysis (Dimensionality Reduction)

Scaling

Most datasets contain features highly varying in magnitudes, units and range.

Most machine learning algorithms have problems with this because they use distance measures or calculate gradients. The features with high magnitudes will weigh in a lot more in the distance calculations than features with low magnitudes and gradients may end up taking a long time or are not accurately calculable.

To overcome this effect, we scale the features to bring them to the same level of magnitudes. The two most discussed scaling methods are Normalization and Standardization.

Normalization typically means to rescale the values into a range of [0,1]:

$$\hat{x}_i = \frac{x_i - x_{min}}{x_{max} - x_{min}}$$

Standardization typically means to rescale data to have a mean of 0 and a standard deviation of 1 (Z scores):

$$\hat{x}_i = \frac{x_i - \mu}{\sigma}$$

If necessary scaling can be inverted:

$$x_i = \hat{x}_i \cdot \left(x_{max} - x_{min}\right) + x_{min} \quad \textbf{resp.} \quad x_i = \hat{x}_i \cdot \sigma + \mu$$

Type Conversion (Encoding)

Many machine learning algorithms cannot work with categorical data directly. To convert categorical data to numbers, there exist two variants:

Label encoding refers to transforming the word labels into numerical form so that the algorithms can understand how to operate on them. Every categorical value is assigned to one numerical value, e.g. young → 1, middle_age → 2, old → 3. This only works in specific situations where you have somewhat continuous-like data, e.g. if the categorical feature is ordinal.

One hot encoding is a representation of a categorical variable as binary vectors. Every categorical value is assigned to an artificial binary variable. If the corresponding categorical value occurs in a data row the value of its binary replacement is equal to 1 else 0, e.g.

Country
Japan
U.S
India
China
U.S
India

Country_Japan	Country_U.S	Country_India	Country_China
1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1
0	1	0	0
0	0	1	0

It is usual when creating dummy variables to have one less variable than the number of categories present to avoid perfect collinearity (dummy variable trap).

TODO

Example of Feature Engineering (I)

Data sets often contain date/time features. These features are rarely useful in their original form because they only contain ongoing values. However, they can be useful for extracting cyclical factors, such as weekly or seasonal effects. Suppose, we are given a data “flight date time vs status”. Then, given the date-time data, we have to predict the status of the flight.

But the status of the flight may depend on the hour of the day, not on the date-time. To analyze this, we will create the new feature “Hour_Of_Day”. Using the “Hour_Of_Day” feature, the machine will learn better as this feature is directly related to the status of the flight.

	Date_Time_Combined	Status
0	2018-02-14 20:40	Delayed
1	2018-02-15 10:30	On Time
2	2018-02-14 07:40	On Time
3	2018-02-15 18:10	Delayed
4	2018-02-14 10:20	On Time

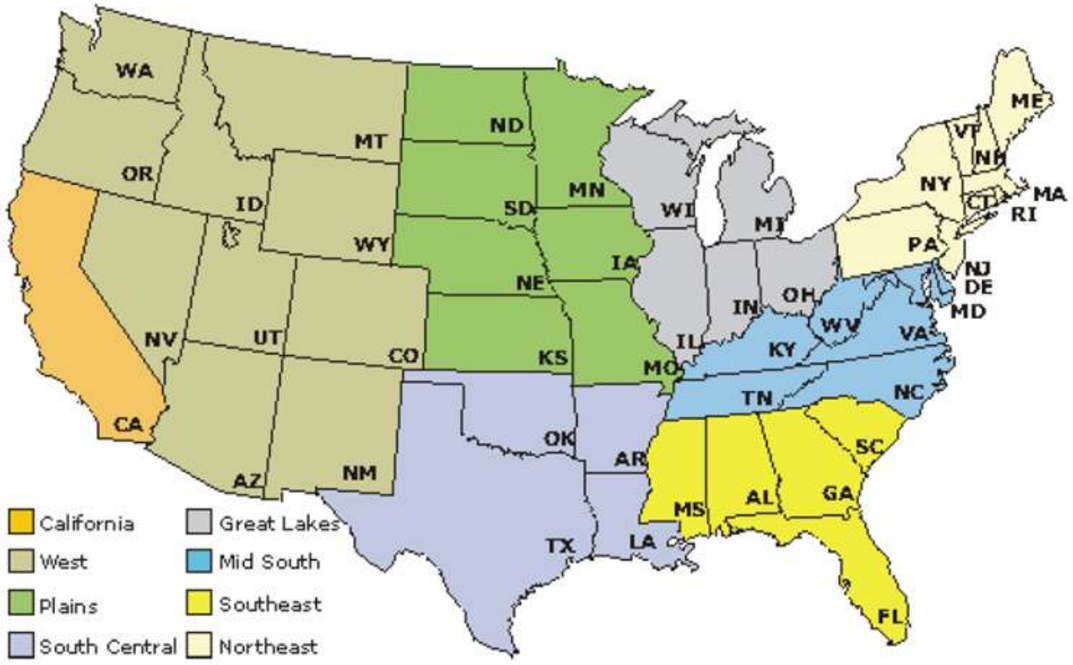
	Hour_Of_Day	Status
0	20	Delayed
1	10	On Time
2	7	On Time
3	18	Delayed
4	10	On Time

— Source: Shekhar, A.: What Is Feature Engineering for Machine Learning?, medium.com.

Example of Feature Engineering (II)

Suppose we are given the latitude, longitude and other data with the objective to predict the target feature “Price_Of_House”. Latitude and longitude are not of use in this context if they are alone. So, we will combine the latitude and the longitude to make one feature.

In other cases, it might be appropriate to transform latitude and longitude into categories which reflect regions, for example.



Example of Feature Engineering (III)

> Suppose we are given a feature “Marital_Status” and other data with the objective to classify customers into “Creditworthy” and “Not_Creditworthy”. In the data set the martial status has many different values, for example:

- single living alone
- single living with his parents
- married living together
- married living separately
- divorced
- divorced but living together
- registered partnerships
- living in marriage-like community
- widowed
- …

> To avoid a transformation into too many and maybe dominating dummy features, we can group the similar classes, e.g. in single, married, widowed.

TODO

> If there exist some remaining sparse classes which cannot be assigned in a meaningful way they can be joined into a single

Partitioning the Data

> The partitioning of the data in **Training and Test Data** has the aim to proof if the analytical results can be generalized. The analysis (e.g. the development of a classifier) is carried out on the basis of training data. Subsequently, the results are applied to the test data. If the results are significantly worse than the training data, the model is not generalizable, which is called overfitting.

> The partitioning of the data in training and test data can be carried out in the following ways:

- By random/stratified/… sampling (problem with the repeatability)
- according to a list
- according to rules (e.g. the first/last 50 records or every twelfth)

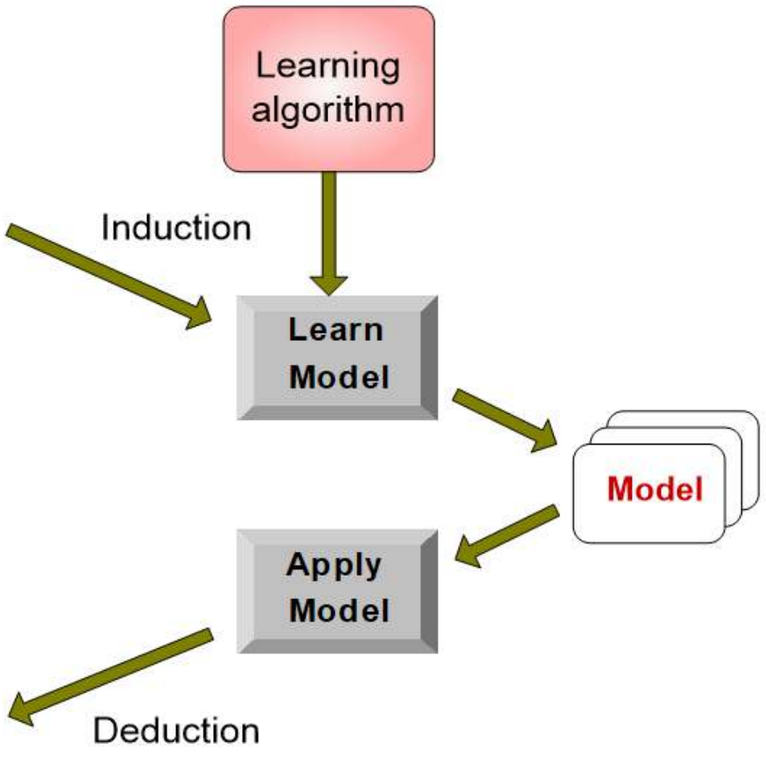
Applying Training and Test Data

Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Training Set

Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?

Test Set



— Source: http://www.cs.kent.edu/~jin/BigData/Lecture10-ML-Classification.pptx

Partitioning

Partitions into training and test data

Example:
Training = 60%
Test = 40%

day.mins	day.calls	day.charge	intl.mins	intl.calls	intl.charge	churn
258.6	84	43.96	11.2	5	3.02	No
129.1	137	21.95	12.7	6	3.43	Yes
156.6	88	26.62	12.3	5	3.32	No
332.9	67	56.59	5.4	9	1.46	Yes
190.7	114	32.42	8.1	3	2.19	No
62.4	89	10.61	5.7	6	1.54	Yes
226.1	105	38.44	10.3	5	2.78	No
249.6	118	42.43	11.8	3	3.19	Yes
126.3	102	21.47	9.4	2	2.54	No
173.1	85	29.43	14.6	15	3.94	Yes